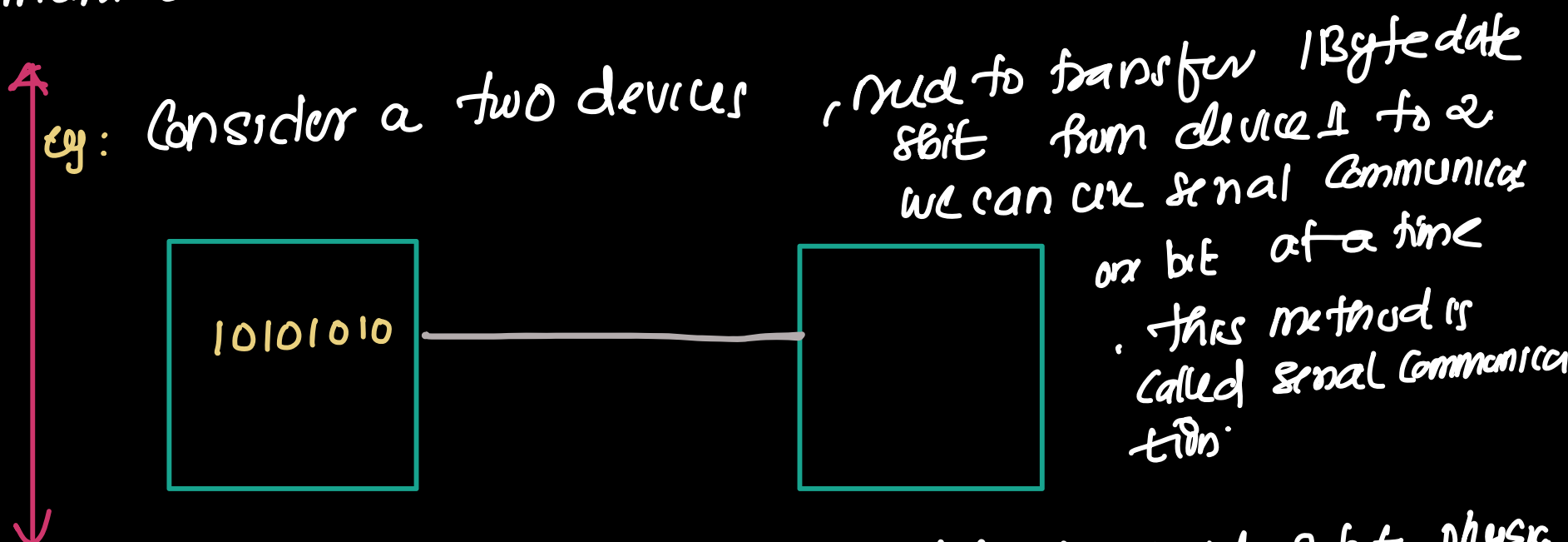


SERIAL I/O

→ Serial I/O is a method of data transfer where data is sent one bit at a time over a single communication line.



→ If we are sending 8 bit parallelly we need 8 bit physical lines. But there is a limitation in No. of physical pins for this kind of data transfer. If we use serial communication we can transmit 8 bit using single wire (pin). Less no. of pins is required. And serial communication provide faster data transfer so serial I/O are popular due to this reason.

There are many serial communication standards are developed for doing serial communication. Microcontroller provide dedicated hardware to easily send data serially via these standards.

Commonly used serial interface standards are

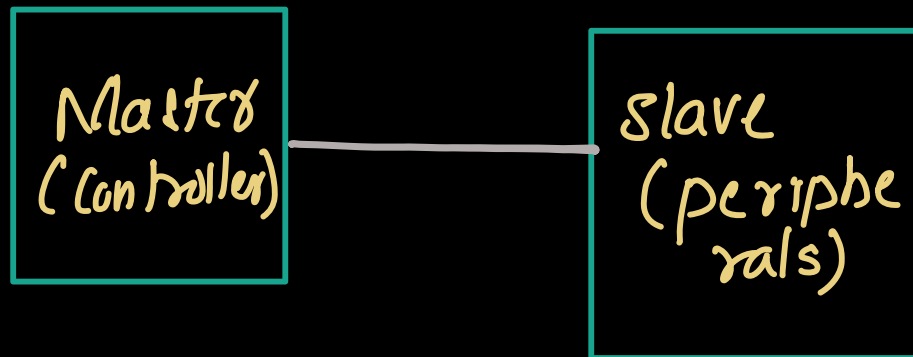
- ① Serial peripheral interface (SPI)
- ② Universal Asynchronous receiver/transmitter UART.

- Serial I/O is a method of data transfer where data is sent one bit at a time over a single communication line.
- Instead of sending 8 bit (1 byte) at once (parallel), because it uses few wires and is fast enough for many applications.
- Microcontroller have a limited number of physical pins. Serial I/O is so popular because it drastically reduces the number of wires/pins required for the communication.
- There are many standards available for transfer data serially between devices. SPI & UART are the two standards used for serial communication.

M SIGMA
GOKULAM

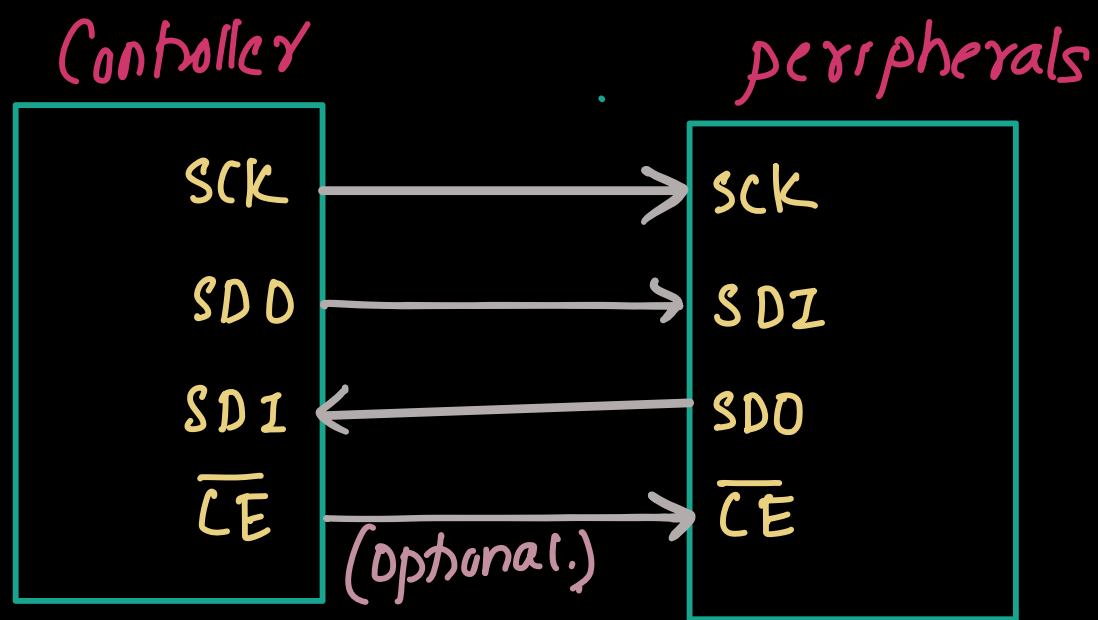
Serial peripheral interface (SPI)

— it is a serial communication protocol used for high speed data transfer b/w a master (controller) and one or more slave (peripherals) devices.



→ An SPI system consists of

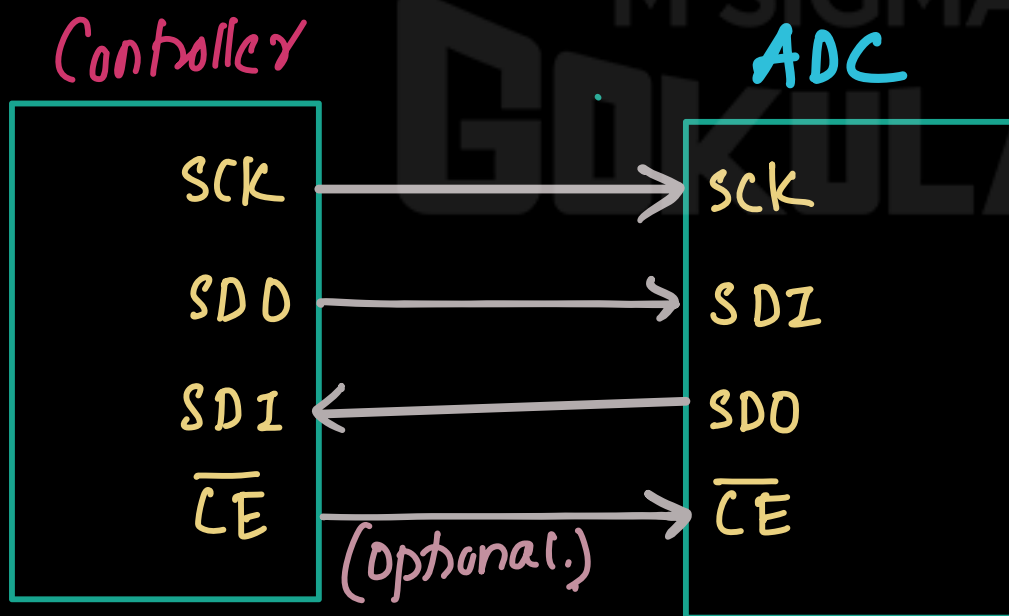
- ① Master (Controller) — Generate clock and control communication
- ② slave (peripherals) — receive data and responds to the master
- ③ Communication Lines
 - ① **SCK** — serial clock — it is generated by the controller to synchronize the data transfer
 - ② Serial data out ^(SDO) — this line used to transmit data from master to slave
 - ③ Serial data input ^(SDI) — this line is used to receive data from slave to master
 - ④ chip enable ($\overline{CE}$) — An active low signal send by the controller to alert the specific peripheral that data is coming.



Working of SPI

Consider a microcontroller based embedded system where we need to send data to a peripheral device, such as Digital to Analog Converter (DAC). DAC require digital input from the microcontroller in order to generate a corresponding analog voltage.

- Microcontroller act as Master
- DAC act as slave

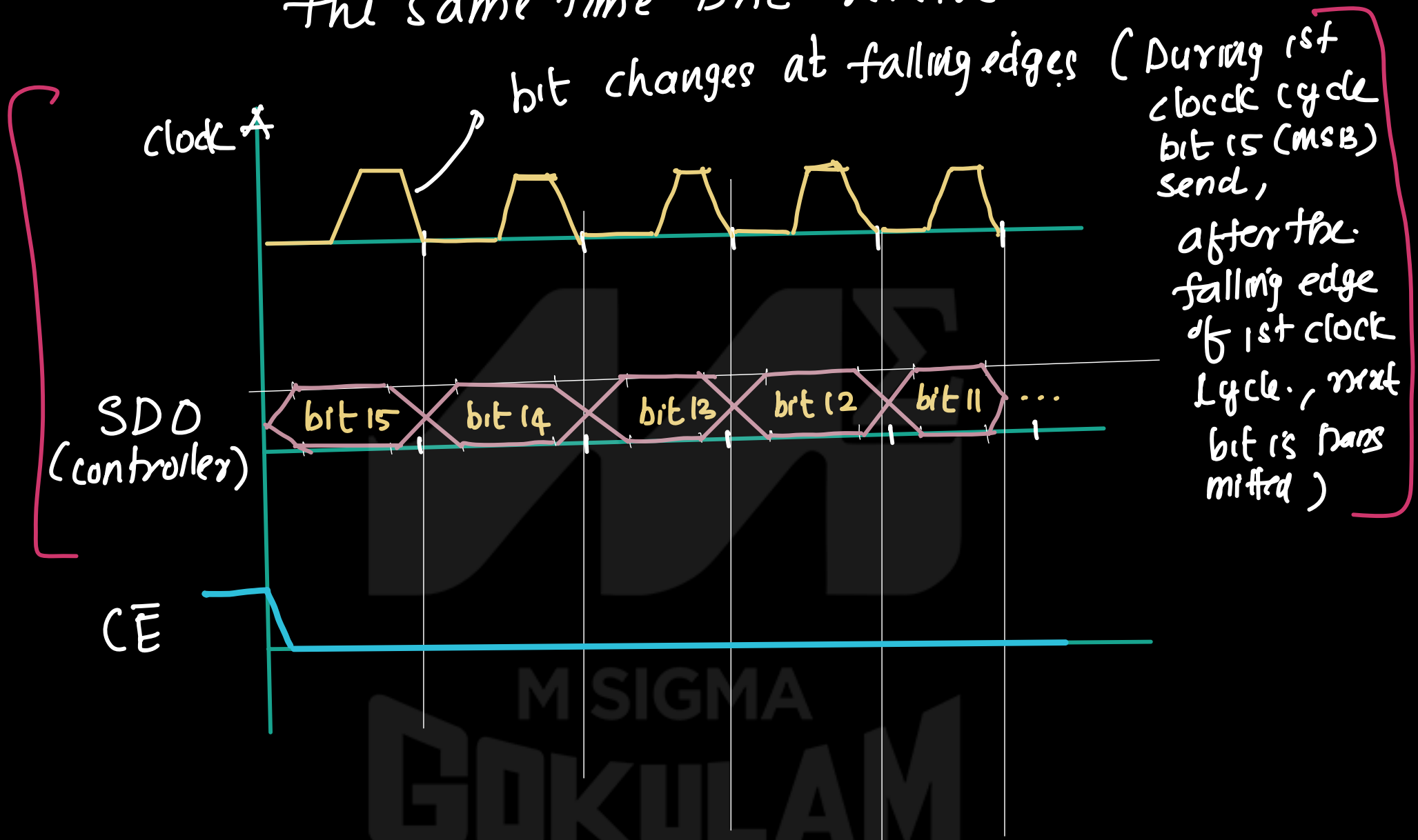


- Step 1 — Microcontroller initiates communication by making chip enable line low. this activates the DAC and prepare it for receive the data
- ② — Microcontroller loads the digital data (Eg 16bit value) into its internal shift register
 - ③ — Microcontroller generate a clock signal (sck) this clock provide synchronization b/w ADC and

controller. Microcontroller send clock signal to the SLK pin of DAC.

④ Controller send the data one bit at a time from SDO to SDI pin of DAC.

⑤ For each clock pulse, one bit data is send from the microcontroller to the DAC. The data transmitted one bit at a time. At the same time DAC receive these bits



— once the entire data packet is transmitted, then master stops the clock pulse and make chip enable line to high.

→ when signal at $\overline{CE}$ is high then DAC process the fully received binary number and update its analog value.

Advantages of SPI

- ① it support full duplex communication.
data can be send in both direction at same time (master to slave & slave to master)
- ② it provide high speed communication
- ③ No internal clock needed for peripherals. Because master generate clock signal, the peripheral devices do not need to generate thier own internal clock to transmit and receive the data.

Sample Questions.

Explain the four hardware signals involved in the SPI connection. For each signal, identify its direction (Master to Slave or vice versa) and explain its specific role in interfacing a Master Microcontroller with a High-Resolution Digital-to-Analog Converter (DAC)

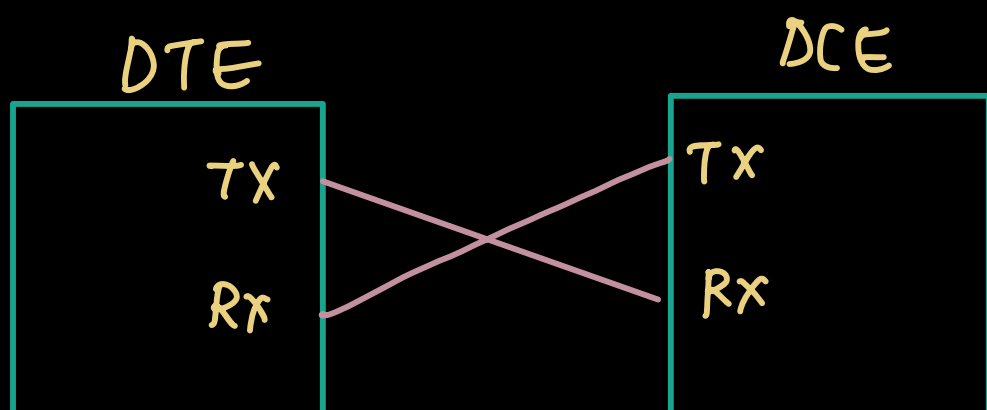
M SIGMA
GOKULAM

Universal Asynchronous receiver/transmitter (UART)

- UART is physical circuit found inside the microcontroller or stand alone ICs.
- its main purpose is to transmit and receive serial data without sending a clock.
- it is a serial I/O peripheral that enables communication b/w two systems without using shared clock pulse. Instead of common clock, both systems are agreed in advance on a data rate (baud rate) and generate their own clock. Because their clocks are not synchronized, the communication is called asynchronous.

Basic Communication set up.

- UART communication occurs b/w two systems
 - ① DTE (Data Terminal equipment) - eg: Microcontroller or Computer
 - ② DCE (Data Communication equipment) - eg: peripherals or modem
- data transfer b/w these two systems happens through two lines
 - ① TX (transmit) → send data
 - ② RX (Receive) → Receives data



Connection method - To communicate, two devices must be cross connected. TX pin of first device connected to the RX pin of second device and RX pin of 1st device is connected to the Tx pin of 2nd device.

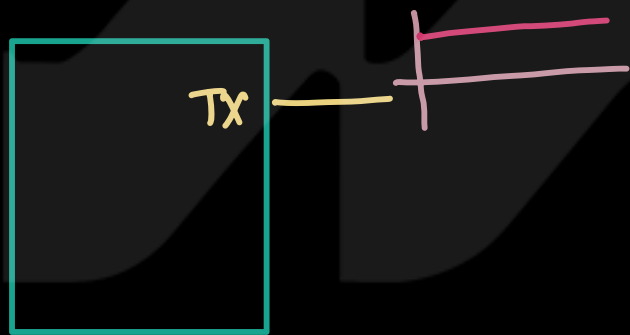
— UART is not used common clock signal. So there is no external clock signal to tell the receiver exactly when to read the incoming bits.

→ So UART hardware package the data in the form of a frame. Single data frame consists of

- ① start bit
- ② data bit
- ③ Parity bit (optional.)
- ④ stop bit.

Start bit:

- this bit is used to indicate the beginning of transmission
- Line (eg: Tx line) is not in use, it rests at high (logic 1) state

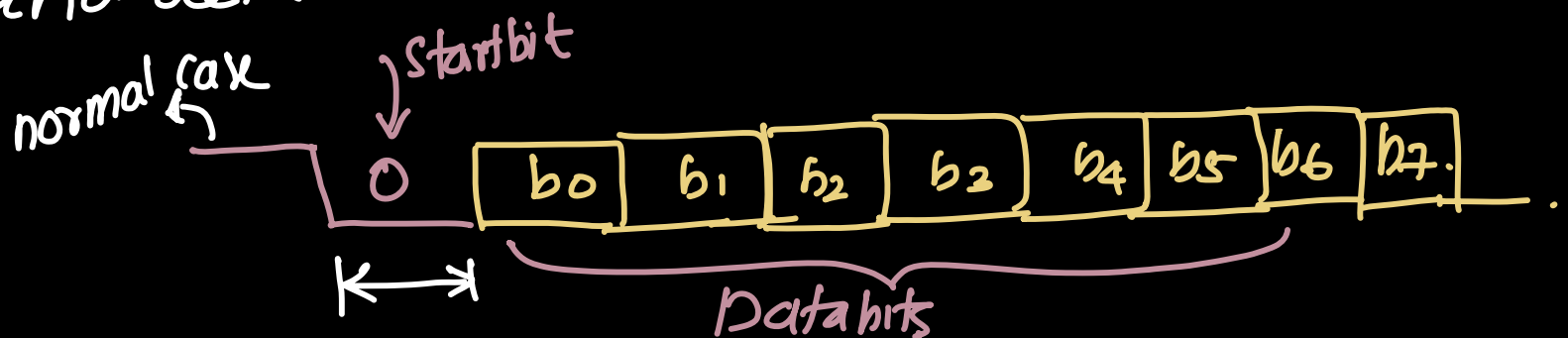


→ to signal the beginning of transmission, transmitter send a start bit '0' through the TX line (a

Transmitter makes signal at TX pin from logic 1 to Logic 0. Receiver detect the falling edge and recognising the data is coming from the transmitter

Data bits

- these bits represents the actual data need to send serially. This is usually 8 bit
- immediately after sending start bit (0), transmitter send actual data bits sequentially.



Parity bit

— After sending data bits, one bit is sent for error detection

e.g. For even parity (data bits consist of even number of 1's). then extra bit is set to 1
For odd parity, extra bit will be '0'

— these bits can be used to check for errors in the received data.

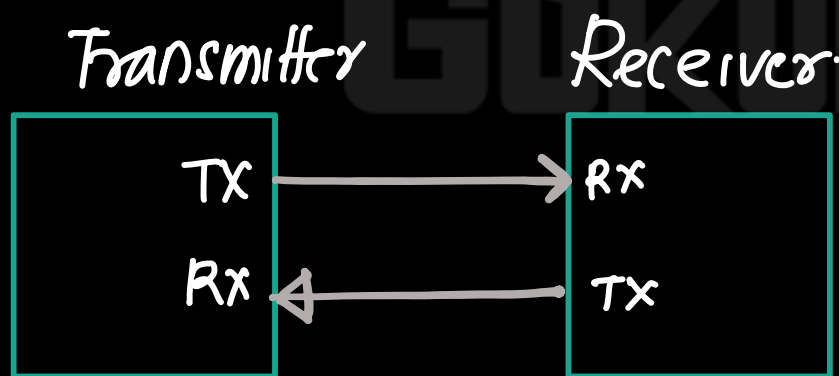
stop bit

— after the parity bit, transmitter sends one last bit to indicate the end of the frame. (bit = 1)



Working

Consider a serial communication system.



transmission

- Communication line remains at logic 1 when no data is being transmitted
- when transmission begins, the transmitter sends a start bit (0)
- receiver detects the transition from Logic 1 to Logic 0
- transmitter sends the data bits one bit at a time at a agreed baud rate.
- the receiver samples each bit at correct time intervals

- If error check bits is enables use that bit to check whether the error is present or not
- Transmitter send last bit (stop bit = 1) to indicate the end of the frame.
- Receiver collect the all the data.

Features

- ① no shared clock (Asynchronous)
- ② use start and stop bit for synchronization
- ③ Support full duplex mode

Disadvantages.

- ① slower than SPI.



Comparison of SPI and UART

SPI

- Synchronous (use shared clock)
- Support full duplex
- standard SPI connection requires 4 lines (SCK, SDO, SDI, $\overline{CE}$)
- provide high speed data transfer

UART

- Asynchronous (no shared clock signal)
- Support full duplex
- Requires only 2 lines (RX, TX)
- slower than SPI

